

Retrieval-Augmented Generation: A Short Overview

Introduction

Retrieval-Augmented Generation, commonly known as RAG, is a technique that combines a document retriever with a generative language model. Instead of relying only on the knowledge a language model learned during training, a RAG system first searches a collection of documents for relevant passages, then provides those passages to the language model as context before it generates an answer. This approach was introduced to reduce hallucination and to allow AI systems to answer questions about specific, up-to-date, or private documents that the model was never trained on.

A RAG system is typically made up of three main components: a document store, an embedding model, and a generative language model. Each of these components plays a distinct role in producing an accurate, grounded answer.

How Retrieval-Augmented Generation Works

The RAG process begins when a document is broken into smaller pieces called chunks. Chunking is necessary because language models can only process a limited amount of text at once, and smaller chunks allow the system to retrieve only the most relevant information rather than an entire document.

Each chunk is converted into a numerical vector using an embedding model. This process is called embedding, and the resulting vector captures the semantic meaning of the text. Two chunks with similar meaning will have embedding vectors that are close together in vector space, even if they do not share the same exact words.

When a user asks a question, the question itself is also converted into an embedding vector. The system then searches for the stored chunks whose vectors are closest to the question's vector. This search process is called similarity search, and it is what allows a RAG system to find relevant information based on meaning rather than exact keyword matches.

Vector Databases and Embeddings

A vector database is a specialized type of database designed to store embedding vectors and perform fast similarity search across potentially millions of entries. ChromaDB is one example of a lightweight, open-source vector database that can run entirely on a local machine without needing a separate server.

Sentence-Transformers is a popular library used to generate embeddings for sentences and paragraphs. Models from this library are trained specifically so that semantically similar sentences end up with similar embedding vectors, which makes them well suited for retrieval tasks in RAG systems.

One important design decision in a RAG system is the chunk size, meaning how much text is grouped together before being embedded. Smaller chunks tend to give more precise retrieval results, since each chunk focuses on a single idea, but very small chunks can lose important surrounding context. Larger chunks preserve more context but may reduce retrieval precision, since irrelevant text can be mixed in with the relevant part.

Local Large Language Models and Ollama

Ollama is a tool that allows large language models to be downloaded and run directly on a personal computer, without sending any data to an external server. This is particularly useful for applications that need to keep data private, or that need to avoid the cost of paid cloud-based AI application programming interfaces.

Phi-3 is a relatively small, open-weight language model that can run efficiently on everyday laptop hardware, including machines without a dedicated graphics processing unit. Despite its smaller size compared to models like GPT-4, Phi-3 is capable of following instructions and generating coherent answers when given relevant context, which makes it a practical choice for a local Retrieval-Augmented Generation system.

When a local language model is combined with retrieved context in a RAG pipeline, the model is not asked to recall facts purely from memory. Instead, it is asked to read the provided passages and use them to answer the question, which is why RAG systems are generally considered more reliable than a language model answering from memory alone.

Evaluating a Retrieval-Augmented Generation System

Evaluating a RAG system involves more than checking whether it produces an answer that sounds correct. A thorough evaluation typically measures retrieval precision, which is whether the system successfully retrieves the document passage that actually contains the answer to the question.

Answer accuracy measures whether the generated answer is factually correct and supported by the retrieved passage. A related concept is faithfulness, which measures whether the generated answer only contains information that is actually present in the retrieved context, rather than information invented by the language model.

Response time is another important evaluation metric, particularly for locally run systems, since the hardware available on a personal computer is generally slower than specialized cloud infrastructure. A well-designed local RAG system should aim to answer most questions within a few seconds to remain practical for everyday use.